1 · Model Predictive Control

1.1 Setup

- Model $x_{k+1} = f(x_k,u_k)$, $f(0,0) = 0$, constraints $x \in \mathbb{X}$, $u \in \mathbb{U}$.
- $x_{k+\ell|k}$ = state at $k+\ell$ predicted at time k ($x_{k|k}$ = measured x_k); $U = [u_{k|k}; \ldots; u_{k+N-1|k}]$ (stacked future inputs).

$$\mathbf{J(x_k,U) = \Sigma_{\ell=0}^{N-1} (x^TQx + u^TRu)_{k+\ell|k} + x^T_{k+N|k} Q_F x_{k+N|k}}$$

- General form $\Sigma L(x,u) + F(x_{k+N|k})$; design knobs L, F, N.

1.2 What Q, R, Q_F do

- **Q ≥ 0** (state weight): penalises distance from target. Bigger ⇒ drives $x \to 0$ faster. Diagonal entries pick which states matter (diag(1,0) = position only).
- **R > 0** (input weight): penalises effort, fuel, actuator wear. Bigger ⇒ smaller, gentler inputs, slower response. $R \succ 0$ ⇒ unique minimiser.
- **Q_F ≥ 0** (terminal weight): penalises where the plan ends; stands in for all cost after the horizon. Chosen from Lyapunov/Riccati ⇒ closed-loop stability.
- Only the **ratio** matters (R = ρI): small ρ ⇒ fast convergence, large inputs; large ρ ⇒ slow, small inputs (input energy J_{in} vs output energy J_{out} trade-off; double integrator ρ = 0.3 vs 10).
- Unstable plant with N = 1 and large ρ can be unstable in closed loop; a longer N or good Q_F fixes it.

1.3 Receding horizon optimisation

- (1) Measure x_k. (2) Solve $U^* = \text{argmin}_{U\in\mathbb{U}^N} J(x_k,U)$. (3) Apply only the first input $u_k = u^*_{k|k} = [I\ 0\ \ldots\ 0]U^*$. (4) k+1: measure, shift window, solve again.
- Result is a feedback law $\mathbf{u_k = \kappa(x_k)}$; in general $u^*_{k+1|k} \neq u^*_{k+1|k+1}$.
- Why: corrects disturbances and model error, handles constraints, looks ahead, finite computation.
- Longer N: better (long-term impact) but more computation; moderate N ≈ infinite horizon. Nonlinear models need numerical optimisation. Integral action: penalise $u_{k+\ell+1|k} - u_{k+\ell|k}$. State + input constraints may make it **infeasible**.

Example: $x_{k+1} = x+u$, N=2, Q=R=Q_F=1, x_0=10. Plan rule $u_{k|k} = -0.6x_k$, $u_{k+1|k} = -0.5x_{k+1|k}$. k=0: plan −6 ($x_{1|0}$=4), −2 ($x_{2|0}$=2); apply −6 only. Wind ⇒ x_1=5 ⇒ re-plan u=−3. Even if x_1=4: $u_{1|1} = -2.4 \neq u_{1|0} = -2$.

1.4 LTI, quadratic cost: explicit solution

- $x_{k+\ell|k} = A^\ell x_k + \Sigma_{j=0}^{\ell-1} A^{\ell-j-1} B u_{k+j|k}$ ⇒ $\mathbf{X = \Phi U + \Lambda x_k}$ (X = stacked predictions $x_{k+1|k}$ … $x_{k+N|k}$).
- Φ = [B 0 … 0; AB B … 0; … ; A^{N-1}B … AB B], Λ = [A; A^2; …; A^N], Q_{blk} = diag(Q,…,Q,Q_F), R = ρI.

$$\mathbf{J = x_k^T(\Lambda^TQ_{blk}\Lambda+Q)x_k + U^T(\Phi^TQ_{blk}\Phi+\rho I)U + 2U^T\Phi^TQ_{blk}\Lambda x_k}$$

$$\mathbf{U^* = -(\Phi^TQ_{blk}\Phi + \rho I)^{-1}\Phi^TQ_{blk}\Lambda\, x_k}$$

$$\mathbf{u_k = -Kx_k,\ K = \text{first row block of } (\Phi^TQ_{blk}\Phi+\rho I)^{-1}\Phi^TQ_{blk}\Lambda}$$

- Uses $\partial(u^TXu)/\partial u = 2Xu$, $\partial(a^Tu)/\partial u = a$. Constant gain = finite-horizon LQR; Hessian $\succ 0$ ($\rho > 0$) ⇒ unique minimum.

Example: a=2, b=1, N=2, Q=Q_F=ρ=1: Φ = [1 0; 2 1], Λ = [2; 4], H = $\Phi^T\Phi$+I = [6 2; 2 2], $\Phi^T\Lambda$ = [10; 4], H^{-1} = ⅛[2 −2; −2 6] ⇒ U^* = −[1.5; 0.5]x_k; apply −1.5x ⇒ $x_{k+1} = 0.5x_k$ (stable).

- **Constraints** ⇒ **QP**: min $U^THU + U^Tf$ s.t. $A_{in}U \le b_{in}$, $A_{eq}U = b_{eq}$; state limits become input limits via $X = \Phi U + \Lambda x_k$ (Kronecker $1_N \otimes$[1 0 0] picks positions).
- Discretise: double integrator A = [1 Δt; 0 1], B = [Δt²/2; Δt]; ZOH [A B; 0 I] = exp([A_c B_c; 0 0]T_s).

1.5 Stability

- **A1**: F(0)=0, F ≥ 0, L(0,0)=0, L(x,u) ≥ α(‖x‖) (α increasing, α(0)=0). **A2**: local κ_f with $F(f(x,\kappa_f(x))) \le F(x) - L(x,\kappa_f(x))$ (terminal cost = Lyapunov function).
- Proof: V_k = optimal cost. Shifted old plan + κ_f at the end is feasible with cost ≤ $V_k - L \ge 0 \le V_{k+1} \le V_k - \alpha(\|x_k\|)$ ⇒ V converges ⇒ $x_k \to 0$.
- Finite input set: R = 0, κ_f = 0, Q_F from $\mathbf{A^TQ_FA + Q = Q_F}$ (A stable; scalar $Q_F = q/(1-a^2)$).

1.6 Vehicle platooning

- Car: $\dot p = v$, $\dot v = a$, $\dot a = -a/\tau + u/\tau$ (τ ≈ 1.5). Gap error $e_{i,k} = x_{i-1,k} - x_{i,k}$, $e_{k+1} = Ae_k + B\bar u_k$, $u_i = u_{i-1} - \bar u_i$.
- LQR (DARE) ignores limits ⇒ car 3 hits car 2, 288 km/h. MPC adds p_{min} = 3 m, 0 ≤ v ≤ 25 m/s, −6 ≤ a ≤ 3 m/s² ⇒ QP keeps gaps and limits.

2 · Deterministic DP

2.1 Principle of optimality

- "The rest of an optimal trajectory, from any intermediate point (however it was reached), is optimal from that point." ⇒ solve the short tail problems first and build longer ones from them.

2.2 Shortest path

- dist(goal) = 0; dist(X) = $\min_Y$[cost(X→Y) + dist(Y)], working backwards from the goal.

Example: E 0; B 2+0 = 2; D min{1+0, 1+2} = 1; A 6+2 = 8; C min{3+1, 4+8} = 4; S min{2+4, 1+8} = 6 ⇒ **S→C→D→E, cost 6**. Greedy S→A (cost 1) is wrong. DP is optimal with no heuristic (unlike A*).

2.3 The DP algorithm

$$\mathbf{J_N(x) = g_N(x);\ J_k(x) = \min_{u\in\mathbb{U}} [\, g_k(x,u) + J_{k+1}(f_k(x,u))\,],\ k = N-1\ \ldots\ 0}$$

- Cost $J(x_0,U) = g_N(x_N) + \Sigma g_k(x_k,u_k)$; $J^* = J_0(x_0)$.
- Forward pass for actions: $u^*_0 = \text{argmin}[g_0 + J_1(f_0)]$, $x^*_1 = f_0(x_0,u^*_0)$, …
- J_k needed for **every** state ⇒ discretise; curse of dimensionality ⇒ topics 5 and 6.

2.4 LQR solved by DP

$$\mathbf{S_N = Q_F;\ S_k = A^TS_{k+1}A + Q - A^TS_{k+1}B(R+B^TS_{k+1}B)^{-1}B^TS_{k+1}A}$$

$$\mathbf{u^*_k = L_kx_k,\ L_k = -(R+B^TS_{k+1}B)^{-1}B^TS_{k+1}A,\ J_k(x) = x^TS_kx,\ J^* = x_0^TS_0x_0}$$

- Why: $J_{N-1} = \min_u[x^TQx + u^TRu + (Ax+Bu)^TS_N(Ax+Bu)]$ is quadratic in u; set d/du = 0, substitute back, repeat by induction.
- Scalar (a,b,q,r): $S_k = q + a^2rS_{k+1}/(r + b^2S_{k+1})$, $L_k = -abS_{k+1}/(r + b^2S_{k+1})$.

Example: a=b=q=r=Q_F=1, N=2: S_2=1, S_1 = 1 + 1/2 = 1.5, S_0 = 1 + 1.5/2.5 = 1.6; L_1 = −0.5, L_0 = −0.6; J^* = 1.6x_0^2. (MPC with N=2 applies L_0 = −0.6 every step.)

3 · Stochastic DP & MDPs

3.1 Policies, not sequences

- $x_{k+1} = f_k(x_k,u_k,w_k)$. Policy π = {$\mu_0,\ldots,\mu_{N-1}$}, $u_k = \mu_k(x_k)$: reacts to the noise that actually happened ⇒ beats fixed sequences (equal when deterministic).
- $J_\pi(x_0) = E[g_N(x_N) + \Sigma g_k(x_k,\mu_k(x_k),w_k)]$; w_k may depend on x_k,u_k, not on earlier w's.

3.2 The stochastic DP algorithm

$$\mathbf{J_N(x) = g_N(x);\ J_k(x) = \min_u E_w[\, g_k(x,u,w) + J_{k+1}(f_k(x,u,w))\,]}$$

- Minimiser $\mu^*_k(x)$ for every x, k ⇒ optimal policy; store it offline, or store J_k and minimise online.
- **Expectation trick** (E w = 0, E $w^2 = \sigma^2$, w uncorrelated with x, u): $\mathbf{E[(a+w)^2] = a^2 + \sigma^2}$. σ^2 doesn't depend on u ⇒ same best u, higher cost. Vector: $E[w^TSw] = tr(S\Sigma)$.

Example (2024 Q2): $x_{k+1} = x+u+w$, $g_k = g_N = \beta x^2$, N=3. $J_3 = \beta x^2$. $J_2 = \beta x^2 + \beta\min_u(x+u)^2 + \beta\sigma^2 \Rightarrow u = -x$, $J_2 = \beta x^2 + \beta\sigma^2$. $J_1 = \beta x^2 + 2\beta\sigma^2$. $J_0 = \beta x^2 + 3\beta\sigma^2$. Pattern $J_k = \beta x^2 + (N-k)\beta\sigma^2$, $\mu_k(x) = -x$. $J_0(x_0) = 1 \Leftrightarrow \beta = 1/(x_0^2 + 3\sigma^2)$.

3.3 LQR with noise and packet dropouts

- Additive noise $x_{k+1} = Ax+Bu+w$: **same gains L_k** (certainty equivalence); cost increases by $\Sigma_k E[w_k^TS_{k+1}w_k]$.
- Dropouts: B_k = B with prob. Γ, 0 with prob. 1−Γ ⇒ E B_k = ΓB, $E[B_k^TSB_k]$ = ΓB^TSB:
  $\mathbf{L_k = -(R + \Gamma B^TS_{k+1}B)^{-1}\Gamma B^TS_{k+1}A}$, $S_k = A^TS_{k+1}A + Q - A^TS_{k+1}\Gamma B(R+\Gamma B^TS_{k+1}B)^{-1}\Gamma B^TS_{k+1}A$. Γ = 1 ⇒ ordinary LQR.

3.4 Markov Decision Processes

- $p_{ij}(u) = P(x_{k+1} = j \mid x_k = i, u_k = u)$, $\Sigma_j p_{ij} = 1$. Any MDP ⇔ $x_{k+1} = w_k$.

$$\mathbf{J_k(i) = \min_u [\, g(i,u) + \Sigma_j\, p_{ij}(u)\, J_{k+1}(j)\,]}$$

Example (Rover): T, R, B; 0 idle, 1 drive. g(T,0)=−3, g(T,1)=−1, g(R/B,0)=0, g(R/B,1)=2. P(0) = [.75 .25 0; 0 0 1; 0 0 1], P(1) = [.8 .2 0; .9 0 .1; 0 .1 .9]. N=3: J_2 = (−3, 0, 0). $J_1(T) = \min\{-3+.75(-3), -1+.8(-3)\} = -5.25$ (idle); $J_1(R) = \min\{0, 2+.9(-3)\} = -0.7$ (drive); $J_1(B) = 0$. J_0 = (−7.1125, −2.725, 0). Policy: T idle; R drive at k=0,1, idle at k=2; B idle (driving at the last step isn't worth it).

4 · Infinite-horizon DP

4.1 Discounted cost

- $J_\pi(x) = \lim_{N\to\infty} E[\Sigma\, \alpha^k g(x_k,\mu_k(x_k),w_k)]$, α ∈ [0,1). α = 0 myopic; α → 1 far-sighted, slower convergence.
- $|g| \le M \Rightarrow |J| \le M/(1-\alpha)$; $\Sigma\alpha^k = 1/(1-\alpha)$ (cost 1 per step, α = 0.9 ⇒ 10); $\Sigma_{k\ge K}\alpha^k = \alpha^K/(1-\alpha)$.
- $J_\pi(x_0) = E[g(x_0,\mu(x_0),w_0)] + \alpha J_\pi(x_1)$. Optimal policy is stationary. Terminating shortest path: α = 1 allowed.

4.2 Bellman's equation

$$\mathbf{J^*(x) = \min_u E[\, g(x,u,w) + \alpha J^*(f(x,u,w))\,];\ J^*(i) = \min_u [\, g(i,u) + \alpha\, \Sigma_j\, p_{ij}(u)\, J^*(j)\,]}$$

- J^* = fixed point of the Bellman operator

TJ = $\min_u E[g + \alpha J(f)]$; the minimiser is μ^*. Fixed μ: $J_\mu(i) = g(i,\mu(i)) + \alpha \Sigma_j p_{ij}(\mu(i)) J_\mu(j)$.

4.3 Value iteration (VI)

- **$V_{k+1}(i) = \min_u[g(i,u) + \alpha \Sigma_j p_{ij}(u) V_k(j)]$**, $V_0 = 0$; matrix $V_{k+1} = \min\{G(0)+\alpha P(0)V_k, G(1)+\alpha P(1)V_k\}$ row by row.
- V_k = optimal k-stage cost; contraction ⇒ $V_k \to J^*$, error ≤ $\alpha^K M/(1-\alpha)$. The greedy policy often settles before the values (not guaranteed).

4.4 Policy iteration (PI)

- (1) **Evaluate** (critic): $J_\mu = g_\mu + \alpha P_\mu J_\mu$ ⇒ **$J_\mu = (I - \alpha P_\mu)^{-1} g_\mu$** (or iterate).
- (2) **Improve** (actor): $\mu^{new}(i) = \text{argmin}_u[g(i,u) + \alpha \Sigma_j p_{ij}(u) J_\mu(j)]$. (3) Policy unchanged ⇒ optimal. Finitely many policies ⇒ finite steps. Rover α=0.96: (0,0,0) → (0,1,0) → (0,1,1).

Example (machine repair): G/D, 0 run, 1 repair; g(G,0)=0, g(G,1)=2, g(D,0)=5, g(D,1)=7; α=0.9; P(0) = [.7 .3; 0 1], P(1) = [.9 .1; .6 .4]. Evaluate (0,0): J(D) = 5 + .9J(D) ⇒ 50; .37J(G) = 13.5 ⇒ 36.49. Improve: G 36.49 vs 36.05 ⇒ repair; D 50 vs 44.70 ⇒ repair. Evaluate (1,1): (26.16, 33.01); improve ⇒ (0,1). Evaluate (0,1): **J* = (20.77, 28.46)**, unchanged ⇒ μ^* = (run when good, repair when damaged). VI from 0: V_1 = (0, 5), V_2 = (1.35, 8.8), V_3 = (3.2265, 10.897).

4.5 VI vs PI

- VI: cheap iteration (one Bellman update), many iterations, values converge only in the limit; gives approx J*, then extract policy.
- PI: expensive iteration (linear solve), few iterations, exact in finite steps; gives the policy directly (actor–critic).
- Variants: truncated/optimistic PI, asynchronous updates (visited states only), average cost lim(1/N)EΣg. Rover: α = 0.9 ⇒ μ = (0,1,0); α = 0.96 ⇒ (0,1,1).
- 2×2 inverse: $[a\ b; c\ d]^{-1} = [d\ -b; -c\ a]/(ad - bc)$.

5 · Approximate DP

5.1 Approximation in value space

$\tilde{u}_k \in \text{argmin}_u$ [$g_k(x_k,u) + J\sim_{k+1}(f_k(x_k,u))$] (J~ = approximate cost-to-go)

- No backward pass: computed forward. J~ = J ⇒ optimal; better J~ ⇒ better policy. J~ from a heuristic, rollout, or a trained model. Loses guaranteed optimality, gains speed.

5.2 Rollout

- For each candidate action: apply it, then run a simple **base (heuristic) policy** to the end; J~ = that total. Pick the cheapest, move one step, repeat.
- **Cost improvement**: rollout is never worse than the base policy (usually better). Name from Tesauro's backgammon.

Example (TSP): from A: AB100 AC300 AD100 AE75 BC50 BD75 BE125 CD100 CE125 DE50. Nearest neighbour: A-E-D-B-C-A = 550. Rollout: stage 1 B 375, C 550, D 625, E 550 ⇒ B; stage 2 C 375, D 650, E 675 ⇒ C; stage 3 D 375, E 425 ⇒ D ⇒ **A-B-C-D-E-A = 375** (optimal here, not always).

5.3 Stochastic problems and certainty-equivalent control

- Approximate both J~ and the expectation. **CEC**: replace w_k by one typical value w_{nom} (usually E w_k) ⇒ deterministic problem $\min_u[g(x,u,w_{nom}) + J\sim(f(x,u,w_{nom}))]$. Exactly optimal for LQG.

Example: $x_1 = x+u+w$, E w = 1, cost $u^2 + x_1^2$. CEC (w = 1): min $u^2 + (x+u+1)^2$ ⇒ u = −(x+1)/2. True expected cost only adds Var(w) ⇒ same minimiser.

5.4 Multistep lookahead and truncated rollout

- ℓ-step lookahead: optimise $u_k, \mu_{k+1} \ldots \mu_{k+\ell-1}$ for $E[\Sigma_{j=k}^{k+\ell-1} g_j + J\sim_{k+\ell}(x_{k+\ell})]$, apply only the first control (= MPC idea). Larger ℓ compensates a poor J~ but costs more.
- Truncated rollout: base policy for $\ell-1$ steps + terminal J~ (J~ = 0 often fine if ℓ large). Infinite horizon: μ~(x) ∈ $\text{argmin}_u E[g + \alpha J\sim(f)]$.

5.5 Parametric approximation

- J~(x,r) with tunable r (polynomials, Gaussian bases, neural nets). Linear: features $\varphi(x) = [\varphi_1 \ldots \varphi_m]^T$ (handcrafted, aggregation), **$J\sim = r^T\varphi(x)$**.
- Train on samples (x^s, β^s): min $\frac{1}{2}\Sigma_s(J\sim(x^s,r) - \beta^s)^2$ (convex if linear). **Least squares: $r = (\Sigma\varphi\varphi^T)^{-1}\Sigma\beta^s\varphi(x^s)$**.
- GD: $r \leftarrow r - \gamma\Sigma_s\varphi(x^s)(r^T\varphi(x^s) - \beta^s)$, γ decreasing; SGD uses one random sample per step. Finite horizon: fit r_k backwards per stage; infinite: one r.

Example: φ = [1, x], (x,β) = (0,1), (1,3), (2,7): $\Sigma\varphi\varphi^T$ = [3 3; 3 5], $\Sigma\beta\varphi$ = [11; 17], inverse ⅛[5 −3; −3 3] ⇒ r = [0.667; 3], J~ = 0.667 + 3x. One GD step from 0, γ = 0.1: gradient −[11; 17] ⇒ r = [1.1; 1.7].

6 · Reinforcement Learning (tabular, model-free)

- Learn from experience (apply u, observe cost g and next state x′) with no model f or p_{ij}. Alternative: identify a model, then use MPC/DP. Also called neuro-dynamic programming.

6.1 Temporal-difference (TD) prediction

$J(x_k) \leftarrow J(x_k) + \delta_k$ [$g(x_k,\mu(x_k),w_k) + \alpha J(x_{k+1}) - J(x_k)$]

- TD target g + αJ(x′) is an unbiased sample of $J_\mu(x)$; the bracket is the **TD error**; δ = learning rate (Robbins–Monro).

Example: J = 0, α = 0.9, δ = 0.5; A (cost 2) → B (cost 0) → A (cost 2) → B: J(A) = 0.5·2 = 1; J(B) = 0.5(0 + 0.9·1) = 0.45; J(A) = 1 + 0.5(2 + 0.9·0.45 − 1) = 1.7025.

6.2 Q-factors

- $Q_\mu(x,u) = E[g(x,u,w) + \alpha J_\mu(f(x,u,w))]$: cost of doing u now, then following μ.
- **$J^*(x) = \min_u Q^*(x,u)$, $\mu^*(x) = \text{argmin}_u Q^*(x,u)$** (no model needed). Q-Bellman: $Q^*(x,u) = E[g + \alpha \min_{u'} Q^*(f(x,u,w),u')]$.

6.3 Sarsa (on-policy)

Q(x,u) ← Q(x,u) + δ [g + αQ(x′,u′) − Q(x,u)]

- Uses (x,u,g,x′,u′) = S,A,R,S,A; u′ = action actually taken next. Loop: pick u (ε-greedy); apply, observe g, x′; pick u′; update; x ← x′, u ← u′. Only the visited entry changes; terminal Q = 0 (target = g).

6.4 Exploration: ε-greedy

- Greedy argmin Q with prob. 1−ε (exploit); random action with prob. ε (explore) ⇒ P(greedy) = 1 − ε + ε/|𝕌|. Decrease ε, e.g. 1/k or max(1/k, 0.01). No exploration ⇒ untried Q stays wrong.

6.5 Q-learning (off-policy)

Q(x,u) ← Q(x,u) + δ [g + α $\min_{u'}$Q(x′,u′) − Q(x,u)]

- Target uses the best next action ⇒ learns Q* while behaving ε-greedily (off-policy). Sarsa learns the values of the policy it follows, exploration included (on-policy) ⇒ more cautious.

Example: Q(s1,a) = 4, Q(s2,a) = 3, Q(s2,b) = 1; observe (s1, a, g=2, s2), exploration picks a; δ = 0.5, α = 0.9. Sarsa: 4 + 0.5(2 + 0.9·3 − 4) = 4.35. Q-learning: 4 + 0.5(2 + 0.9·1 − 4) = 3.45.

6.6 Convergence conditions

- Every (x,u) visited infinitely often; $\delta_k > 0$, **$\Sigma\delta_k = \infty$, $\Sigma\delta_k^2 < \infty$** (1/k ✓, constant ✗, $1/k^2$ ✗).
- Sarsa reaches Q* only if ε → 0 (greedy in the limit); constant ε ⇒ Q of the ε-greedy policy. Rover runs: δ = 0.1, $\varepsilon_k = \max(1/k, 0.01)$, 500 episodes × 1000 steps.

6.7 Bigger problems and caveats

- Use Q(x,u;r) (e.g. a neural net), minimise $E[(g + \alpha \min_u Q(x',u;r) - Q(x,u;r))^2]$ by GD (deep RL, DQN). Learning convergence ≠ closed-loop stability; use known structure.
- **Rewards instead of costs**: best action = max Q (or cost = −reward); Sarsa update unchanged; Q-learning uses max.

Example (2024 Q3): (Sarsa δ = 0.1, α = 0.5; ep1 1A, 2A, 4B, 4B; ep2 1B, 3A, 2A, 4B): Q(2,A) = 0.1·100 = 10; Q(4,B) = 0.1(100 + 0) = 10; Q(3,A) = 0.1(0 + 0.5·10) = 0.5; Q(2,A) = 10 + 0.1(100 + 0.5·10 − 10) = 19.5; rest 0. Tie at 1 ⇒ r + α·max Q(next): A 9.75 > B 0.25 ⇒ **1→A→2→A→4**.

Symbols

x, u, w	state, control, random noise
f, g, g_N	model, stage cost, final cost
N, ℓ	horizon, lookahead steps
$x_{k+\ell\|k}$	prediction of time k+ℓ made at k
Q, R, Q_F	state, input, terminal weights
U, X	stacked future inputs, predictions
Φ, Λ, Q_{blk}	batch matrices, stacked weights
S_k, L_k, K	Riccati matrix, feedback gains
$J_k(x)$, J*	cost-to-go, optimal cost
μ, π, μ*	policy, policy sequence, optimal
$p_{ij}(u)$	probability i → j under u
α	discount factor, 0 ≤ α < 1
T, T_μ, V_k	Bellman operators, VI iterate
J~, φ, r	approx. cost, features, weights
β, γ	training target, step size
Q(x,u)	Q-factor (action value)
δ, ε	learning rate, exploration prob.
Γ, σ^2	delivery prob., noise variance
κ(x)	MPC feedback law